

MANUAL COMPLETO DE ALGORITMOS Y ESTRUCTURAS DE DATOS

Repositorio de Código de Referencia para Programación Competitiva (ICPC)

Este compendio contiene el desglose analítico completo de todas las herramientas algorítmicas de tu repositorio. Cada sección detalla la teoría del algoritmo, su rendimiento en tiempo y espacio, y el código estructurado con comentarios posicionales rigurosos.

1. Búsquedas e Inteligencia Combinatoria

1.1 Búsqueda Binaria (bs_icpc.cpp)

Complejidad Temporal: $O(\log n)$ | **Espacial:** $O(1)$

Reduce el espacio de búsqueda a la mitad en cada paso iterativo. Su variante avanzada de optimización, *Binary Search on Answer*, evalúa la viabilidad de una solución hipotética mediante predicados monótonos.

```
#include <bits/stdc++.h>
using namespace std;

// Búsqueda binaria clásica
int binary_search(vector& arr, int target) {
    int lo = 0, hi = (int)arr.size() - 1; // DONDE: Definición del espacio indexado.
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2; // QUÉ HACE: Calcula el pivote medio evitando desbordamiento.
        if (arr[mid] == target) return mid; // DONDE: Condición de éxito.
        else if (arr[mid] < target) lo = mid + 1; // QUÉ HACE: Recorta la mitad izquierda si el valor
        es menor.
        else hi = mid - 1; // QUÉ HACE: Recorta la mitad derecha si el valor es mayor.
    }
    return -1; // Retorna si el elemento no fue localizado.
}

// Binary Search on Answer
bool feasible(int speed, vector& piles, int T) {
    long long hours = 0; // DONDE: Acumulador de métrica del problema.
    for (int p : piles) hours += (p + speed - 1) / speed; // QUÉ HACE: Techo matemático de la
    división.
    return hours <= T; // Evalúa si la velocidad cumple con la restricción de tiempo.
}

int min_speed(vector& piles, int T) {
    int lo = 1, hi = *max_element(piles.begin(), piles.end()); // Espacio de búsqueda de respuestas.
    while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (feasible(mid, piles, T)) hi = mid; // QUÉ HACE: Si es posible, intenta minimizar cerrando
        por la derecha.
        else lo = mid + 1; // QUÉ HACE: Si no es posible, incrementa el umbral inferior.
    }
    return lo;
}
```

1.2 Exploración Exhaustiva (backtrack_icpc.cpp)

Complejidad Temporal: $O(n!)$ o $O(2^n)$ | **Espacial:** $O(n)$ recursivo

Explora todo el espacio de configuraciones posibles usando llamadas recursivas y podas de estados no válidos (diseño de soluciones de fuerza bruta inteligente).

```
#include <bits/stdc++.h>
using namespace std;

// Permutaciones de un arreglo
void permutations(vector& arr, int start, vector& res) {
    if (start == (int)arr.size()) { res.push_back(arr); return; } // DONDE: Caso base de permutación completa.
    for (int i = start; i < (int)arr.size(); i++) {
        swap(arr[start], arr[i]); // QUÉ HACE: Elige el elemento actual para la posición 'start'.
        permutations(arr, start + 1, res); // Recurrencia.
        swap(arr[start], arr[i]); // DONDE: Backtracking. Deshace el cambio para la siguiente rama.
    }
}

// N-Reinas en tablero N x N
int nQueens(int n) {
    int count = 0;
    vector cols(n, 0), diag1(2*n, 0), diag2(2*n, 0); // DONDE: Vectores de control de amenazas de líneas.
    function solve = [&](int row) {
        if (row == n) { count++; return; } // Caso base: todas las reinas colocadas con éxito.
        for (int col = 0; col < n; col++) {
            if (cols[col] || diag1[row-col+n] || diag2[row+col]) continue; // DONDE: Poda de ramas no válidas.
            cols[col] = diag1[row-col+n] = diag2[row+col] = 1; // QUÉ HACE: Marca la columna y diagonales ocupadas.
            solve(row + 1); // Avanza a la siguiente fila.
            cols[col] = diag1[row-col+n] = diag2[row+col] = 0; // QUÉ HACE: Desmarca para la siguiente opción.
        }
    };
    solve(0);
    return count;
}
```

2. Algoritmos en Grafos y Redes de Flujo

2.1 Búsqueda en Anchura (bfs_icpc.cpp)

Complejidad Temporal: $O(V + E)$ | **Espacial:** $O(V)$

Explora un grafo de manera radial, nivel por nivel. Es la herramienta por excelencia para encontrar caminos mínimos en redes cuyas aristas no tienen ponderación (peso homogéneo).

```
#include <bits/stdc++.h>
using namespace std;

vector bfs(int src, int n, vector<vector>& adj) {
    vector dist(n, -1); // DONDE: Inicialización de distancias. (-1 indica no visitado).
    queue q;
    dist[src] = 0; // Caso origen.
    q.push(src); // DONDE: Cola de control de frontera de exploración FIFO.
    while (!q.empty()) {
        int u = q.front(); q.pop(); // Extrae el nodo procesado en este paso.
        for (int v : adj[u]) {
            if (dist[v] == -1) { // DONDE: Verificación de nodo libre.
                dist[v] = dist[u] + 1; // QUÉ HACE: Asigna la distancia mínima exacta.
                q.push(v); // Empuja a la cola para evaluar sus vecinos posteriormente.
            }
        }
    }
    return dist;
}
```

2.2 Búsqueda en Profundidad (dfs_icpc.cpp)

Complejidad Temporal: $O(V + E)$ | **Espacial:** $O(V)$ por stack

Explora de manera vertical penetrando en la red hasta alcanzar hojas antes de retroceder. Útil para verificar conectividad, detectar ciclos o cálculo de componentes conexas.

```
#include <bits/stdc++.h>
using namespace std;

vector visited;

// Versión Recursiva implícita en Stack del Sistema
void dfs(int u, vector<vector>& adj) {
    visited[u] = true; // DONDE: Registro de visita para evitar ciclos infinitos.
    for (int v : adj[u]) {
        if (!visited[v]) dfs(v, adj); // QUÉ HACE: Transición profunda recursiva inmediata.
    }
}

// Versión Iterativa Explícita
void dfs_iter(int src, int n, vector<vector>& adj) {
    vector vis(n, false);
    stack st; // DONDE: Uso de pila LIFO para control explícito de memoria.
    st.push(src);
    while (!st.empty()) {
        int u = st.top(); st.pop();
        if (vis[u]) continue;
        vis[u] = true; // QUÉ HACE: Procesa el nodo al salir de la pila.
        for (int v : adj[u]) {
            if (!vis[v]) st.push(v); // Coloca los vecinos no visitados en el tope.
        }
    }
}
```

2.3 Caminos Cortos Monótonos (dijkstra_icpc.cpp)

Complejidad Temporal: $O((V + E) \log V)$ | **Espacial:** $O(V)$

Encuentra el camino mínimo desde una fuente única en grafos ponderados con costos estrictamente no negativos. Utiliza una cola de prioridad para refinar la arista óptima en cada ciclo.

```
#include <bits/stdc++.h>
using namespace std;
typedef pair pii;

vector dijkstra(int src, int n, vector<pii>& adj) {
    vector dist(n, 1e9); // DONDE: Inicialización con infinito conceptual.
    priority_queue<pii, greater<pii>> pq; // Min-Heap de pares {distancia, nodo}.
    dist[src] = 0;
    pq.push({0, src});
    while (!pq.empty()) {
        auto [d, u] = pq.top(); pq.pop(); // DONDE: Extracción del estado de menor costo actual.
        if (d > dist[u]) continue; // QUÉ HACE: Ignora estados obsoletos (poda de optimización).
        for (auto [v, w] : adj[u]) {
            if (dist[u] + w < dist[v]) { // DONDE: Condición de Relajación de aristas.
                dist[v] = dist[u] + w; // QUÉ HACE: Actualiza con un costo menor descubierto.
                pq.push({dist[v], v}); // Inserta en la cola para propagar la mejora.
            }
        }
    }
    return dist;
}
```

2.4 Costos Negativos (bellman_icpc.cpp)

Complejidad Temporal: $O(V \times E)$ | **Espacial:** $O(V)$

Resuelve caminos mínimos admitiendo pesos negativos y cuenta con la capacidad analítica de reportar ciclos infinitos negativos.

```
#include <bits/stdc++.h>
using namespace std;
struct Edge { int u, v, w; };

vector bellman_ford(int src, int n, vector<Edge>& edges) {
    vector dist(n, 1e9);
    dist[src] = 0;
    for (int i = 0; i < n - 1; i++) { // DONDE: Bucle de relajación estricta V-1 veces.
        for (auto& [u, v, w] : edges) {
            if (dist[u] != 1e9 && dist[u] + w < dist[v])
                dist[v] = dist[u] + w; // QUÉ HACE: Relaja todas las aristas del grafo.
        }
    }
    // DONDE: Iteración V para auditoría de consistencia.
    for (auto& [u, v, w] : edges) {
        if (dist[u] != 1e9 && dist[u] + w < dist[v]) {
            cout << "Ciclo negativo detectado!" << endl; // QUÉ HACE: Reporta inestabilidad si sigue
            bajando el costo.
            return {};
        }
    }
    return dist;
}
```

2.5 Caminos Totales (floyd_icpc.cpp)

Complejidad Temporal: $O(V^3)$ | **Espacial:** $O(V^2)$

Calcula los costos mínimos de viaje entre todos los pares posibles de nodos mediante programación dinámica matricial directa.

```
#include <bits/stdc++.h>
using namespace std;

void floyd_warshall(int n, vector>& dist) {
    for (int k = 0; k < n; k++) // DONDE: Bucle externo obligatoriamente K (nodo intermediario).
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                if (dist[i][k] != 1e9 && dist[k][j] != 1e9)
                    dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]); // QUÉ HACE: Transición DP.
}
```

3. Estructuras de Datos Avanzadas

3.1 Árboles de Segmentos (segment_icpc.cpp)

Complejidad Temporal: Construcción: $O(n)$, Consulta/Update: $O(\log n)$

Árbol asociativo binario para consultas de rangos variables dinámicos con actualizaciones puntuales constantes en el tiempo.

```
#include <bits/stdc++.h>
using namespace std;

struct SegTree {
    int n; vector tree;
    SegTree(int n) : n(n), tree(4*n, 0) {} // Reserva de espacio estándar de 4*N.

    void build(vector& arr, int node, int lo, int hi) {
        if (lo == hi) { tree[node] = arr[lo]; return; } // DONDE: Hojas del árbol.
        int mid = (lo + hi) / 2;
        build(arr, 2*node, lo, mid); // QUÉ HACE: Divide por la izquierda.
        build(arr, 2*node+1, mid+1, hi); // QUÉ HACE: Divide por la derecha.
        tree[node] = tree[2*node] + tree[2*node+1]; // DONDE: Fusión de resultados ascendente.
    }

    void update(int node, int lo, int hi, int idx, int val) {
        if (lo == hi) { tree[node] = val; return; }
        int mid = (lo + hi) / 2;
        if (idx <= mid) update(2*node, lo, mid, idx, val); // QUÉ HACE: Navega a la rama objetivo.
        else update(2*node+1, mid+1, hi, idx, val);
        tree[node] = tree[2*node] + tree[2*node+1]; // Recalcula los rangos modificados.
    }

    long long query(int node, int lo, int hi, int l, int r) {
        if (r < lo || hi < l) return 0; // DONDE: Fuera de rango totalmente.
        if (l <= lo && hi <= r) return tree[node]; // DONDE: Rango cubierto por completo.
        int mid = (lo + hi) / 2;
        return query(2*node, lo, mid, l, r) + query(2*node+1, mid+1, hi, l, r);
    }
};
```

3.2 Árbol Indexado Binario (fenwick_icpc.cpp)

Complejidad Temporal: $O(\log n)$ | Espacial: $O(n)$

Estructura ultraligera basada en operaciones del bit menos significativo (LSB) para resolver sumas de prefijos eficientemente.

```
#include <bits/stdc++.h>
using namespace std;

struct BIT {
    int n; vector bit;
    BIT(int n) : n(n), bit(n+1, 0) {}

    void update(int i, long long delta) {
        for (++i; i <= n; i += i & -i) // QUÉ HACE: Navega sumando el bit LSB (i & -i) hacia arriba.
            bit[i] += delta; // DONDE: Modificación de los rangos contenedores.
    }

    long long query(int i) {
        long long s = 0;
        for (++i; i > 0; i -= i & -i) // QUÉ HACE: Descompone removiendo bits encendidos hacia abajo.
            s += bit[i]; // DONDE: Acumulación de prefijo.
        return s;
    }
    long long query(int l, int r) { return query(r) - (l > 0 ? query(l-1) : 0); }
};
```

4. Conectividad e Infraestructura de Árboles

4.1 Disjoint Set Union (dsu_icpc.cpp)

Complejidad Temporal: $O(\alpha(n))$ quasi-constante | **Espacial:** $O(n)$

Mantiene el control de subconjuntos disjuntos óptimos integrando compresión de caminos y unión por rangos.

```
#include <bits/stdc++.h>
using namespace std;

struct DSU {
    vector par, rank_;
    DSU(int n) : par(n), rank_(n, 0) { iota(par.begin(), par.end(), 0); } // Cada elemento es su propio padre.

    int find(int x) {
        if (par[x] != x) par[x] = find(par[x]); // DONDE: Path Compression.
        return par[x]; // QUÉ HACE: Aplana la estructura apuntando directo a la raíz del conjunto.
    }

    bool unite(int x, int y) {
        x = find(x); y = find(y);
        if (x == y) return false; // DONDE: Detección de pertenencia compartida (ciclo).
        if (rank_[x] < rank_[y]) swap(x, y); // DONDE: Union by Rank.
        par[y] = x; // QUÉ HACE: Cuelga el árbol más pequeño del más grande.
        if (rank_[x] == rank_[y]) rank_[x]++;
        return true;
    }
};
```

4.2 Kruskal MST (kruskal_icpc.cpp)

Complejidad Temporal: $O(E \log E)$ | **Espacial:** $O(V + E)$

Encuentra el árbol de expansión mínima utilizando una estrategia ávida apoyada sobre la estructura DSU.

```
#include <bits/stdc++.h>
using namespace std;
struct Edge { int u, v, w; };

// Usa el DSU definido arriba para validar ciclos en tiempo quasi-constante
int kruskal(int n, vector& edges) {
    // DONDE: Estrategia Greedy fundamental.
    sort(edges.begin(), edges.end(), [](auto& a, auto& b){ return a.w < b.w; }); // QUÉ HACE: Ordena
    de menor a mayor peso.
    DSU dsu(n);
    int mst_cost = 0, edges_count = 0;
    for (auto& [u, v, w] : edges) {
        if (dsu.unite(u, v)) { // DONDE: Intento de unión sin generar ciclos.
            mst_cost += w; // QUÉ HACE: Absorbe el peso de la arista válida.
            if (++edges_count == n - 1) break;
        }
    }
    return mst_cost;
}
```

4.3 Descomposición de Centroides (centroid_icpc.cpp)

Complejidad Temporal: $O(n \log n)$ | Espacial: $O(n)$

Divide y vencerás sobre árboles no lineales encontrando nodos divisores balanceados para resolver consultas complejas sobre trayectorias.

```
#include <bits/stdc++.h>
using namespace std;

vector adj[100005]; int sz[100005]; bool removed[100005];

void calc_size(int u, int p) {
    sz[u] = 1;
    for (int v : adj[u]) {
        if (v != p && !removed[v]) { calc_size(v, u); sz[u] += sz[v]; } // DONDE: Cálculo posicional
    } // dinámico del tamaño.
}

int find_centroid(int u, int p, int tree_sz) {
    for (int v : adj[u]) {
        if (v != p && !removed[v] && sz[v] > tree_sz / 2)
            return find_centroid(v, u, tree_sz); // QUÉ HACE: Se desplaza al subárbol mayoritario
    } // (pesado).
    return u; // DONDE: Nodo centroide localizado (ningún subárbol remanente supera N/2).
}
```

5. Programación Dinámica Avanzada y Matemáticas

5.1 Problema de la Mochila (dp_basic_icpc.cpp)

Complejidad Temporal: $O(n \times W)$ | Espacial: $O(W)$ optimizado

Maximización del valor bajo restricciones de capacidad estricta en sus variantes 0/1 e ilimitada.

```
#include <bits/stdc++.h>
using namespace std;

int knapsack(int W, vector& wt, vector& val) {
    vector dp(W + 1, 0); // DONDE: Optimización de espacio en una sola fila.
    for (int i = 0; i < (int)wt.size(); i++)
        for (int w = W; w >= wt[i]; w--) // DONDE: Recorrido inverso para asegurar uso único por
            elemento (Mochila 0/1).
            dp[w] = max(dp[w], dp[w - wt[i]] + val[i]); // Transición clásica DP.
    return dp[W];
}

int unbounded_knapsack(int W, vector& wt, vector& val) {
    vector dp(W + 1, 0);
    for (int w = 1; w <= W; w++) // DONDE: Recorrido directo ascendente.
        for (int i = 0; i < (int)wt.size(); i++)
            if (wt[i] <= w) dp[w] = max(dp[w], dp[w - wt[i]] + val[i]); // QUÉ HACE: Permite múltiples
            elecciones del mismo ítem.
    return dp[W];
}
```

5.2 Exponenciación de Matrices (matrix_icpc.cpp)

Complejidad Temporal: $O(k^3 \log n)$ | **Espacial:** $O(k^2)$

Eleva una matriz de transiciones de recurrencias lineales a potencias masivas de orden logarítmico.

```
#include <bits/stdc++.h>
using namespace std;
typedef vector<vector> Matrix;

Matrix multiply(const Matrix& A, const Matrix& B) {
    int n = A.size(); Matrix C(n, vector(n, 0));
    for (int i = 0; i < n; i++)
        for (int k = 0; k < n; k++) if (A[i][k]) // Poda si la celda origen es cero.
            for (int j = 0; j < n; j++) C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % 1000000007; //
    Multiplicación clásica.
    return C;
}

Matrix mat_pow(Matrix A, long long p) {
    int n = A.size(); Matrix res(n, vector(n, 0));
    for (int i = 0; i < n; i++) res[i][i] = 1; // DONDE: Creación de la matriz Identidad.
    while (p > 0) {
        if (p & 1) res = multiply(res, A); // QUÉ HACE: Absorbe la base si el bit binario del
        exponente está encendido.
        A = multiply(A, A); // QUÉ HACE: Eleva la base al cuadrado.
        p >>= 1; // Desplazamiento de bits.
    }
    return res;
}
```

6. Procesamiento de Texto y Cadenas

6.1 String Hashing (hashing_icpc.cpp)

Complejidad Temporal: Preproceso: $O(n)$, Consulta: $O(1)$

Codifica subcadenas completas en firmas numéricas utilizando dobles módulos primos concurrentes.

```

#include <bits/stdc++.h>
using namespace std;

struct StringHash {
    long long MOD1 = 1e9+7, MOD2 = 1e9+9; long long B1 = 131, B2 = 137;
    vector h1, h2, pw1, pw2;

    StringHash(string& s) {
        int n = s.size(); h1.assign(n+1, 0); h2.assign(n+1, 0); pw1.assign(n+1, 1); pw2.assign(n+1,
1);
        for (int i = 0; i < n; i++) {
            h1[i+1] = (h1[i]*B1 + s[i]) % MOD1; // DONDE: Construcción del prefijo hash hash modular
1.
            h2[i+1] = (h2[i]*B2 + s[i]) % MOD2; // DONDE: Construcción del prefijo hash hash modular
2.
            pw1[i+1] = pw1[i]*B1 % MOD1; pw2[i+1] = pw2[i]*B2 % MOD2;
        }
    }

    pair get(int l, int r) {
        long long v1 = (h1[r+1] - h1[l]*pw1[r-l+1] % MOD1 + MOD1) % MOD1; // QUÉ HACE: Extrae el hash
en O(1) restando prefijos.
        long long v2 = (h2[r+1] - h2[l]*pw2[r-l+1] % MOD2 + MOD2) % MOD2;
        return {v1, v2}; // Retorna par ordenado libre de colisiones estándar.
    }
};

```

7. Nuevos Bloques de Programación Dinámica Solicitados

A continuación se inyectan las dos soluciones de DP avanzado requeridas para la expansión de tu biblioteca impresa.

7.1 Digit DP (digit_dp_icpc.cpp)

Complejidad Temporal: $O(L \times 10 \times 2 \times \text{Estado})$ | **Espacial:** $O(L \times 2 \times \text{Estado})$

```

#include <bits/stdc++.h>
using namespace std;

long long dp[20][2][200]; string num_limit;

long long solve_digit_dp(int pos, bool is_less, int sum_digits) {
    if (pos == (int)num_limit.size()) return sum_digits == 7; // DONDE: Caso Base posicional final.
    if (dp[pos][is_less][sum_digits] != -1) return dp[pos][is_less][sum_digits]; // DONDE: Control de
memoization.

    long long answer = 0;
    int limit = is_less ? 9 : (num_limit[pos] - '0'); // QUÉ HACE: Acota las opciones según el prefijo
de borde.

    for (int digit = 0; digit <= limit; digit++) {
        bool next_less = is_less || (digit < limit);
        answer += solve_digit_dp(pos + 1, next_less, sum_digits + digit); // Transición profunda.
    }
    return dp[pos][is_less][sum_digits] = answer;
}

```

7.2 Bitmask DP (bitmask_dp_icpc.cpp)

Complejidad Temporal: $O(n^2 \times 2^n)$ | **Espacial:** $O(n \times 2^n)$

```

#include <bits/stdc++.h>
using namespace std;

int dist[20][20]; int memo[20][1 << 20]; int num_nodes;

int tsp(int u, int mask) {
    if (mask == (1 << num_nodes) - 1) return dist[u][0]; // DONDE: Cierre de ciclo Hamiltoniano
    completo.
    if (memo[u][mask] != -1) return memo[u][mask];

    int min_cost = 1e9;
    for (int v = 0; v < num_nodes; v++) {
        if ((mask & (1 << v)) == 0) { // OPERACIÓN BITWISE: Valida que el nodo 'v' no esté encendido.
            int cost = dist[u][v] + tsp(v, mask | (1 << v)); // OPERACIÓN BITWISE: Enciende el bit
            'v'.
            min_cost = min(min_cost, cost);
        }
    }
    return memo[u][mask] = min_cost;
}

```